CS 10: Operating Systems Laboratory – Assessment Task 9

Investigating Threads

Name: Rodnee Glen A. Martin

Year & Section: 3 BSCS A

Simulator: YASMIN CPU-OS Simulator 7.5.50

Date Performed: October 23, 2026

LO1: Write a source code that creates threads

```
program ThreadTest1
  sub thread1 as thread
    writeln("In thread1")
    while true
    wend
  end sub

  sub thread2 as thread
    call thread1
    writeln("In thread2")
    while true
    wend
  end sub

  call thread2
  writeln("In main")

  do
  loop
end
```

Briefly explain what the above code is doing:

The program declares two subroutines marked “as thread”, so each call starts a new thread instead of a normal subroutine call. The main program calls thread2, which starts thread2 as a child thread, and then main prints “In main” and loops forever. Thread2 calls thread1, which starts thread1 as a child of thread2, and then thread2 prints “In thread2” and loops forever. Thread1 prints “In thread1” and loops forever. In the end three things run at the same time: the main process, thread2 and thread1.

List the order in which you expect the text to be displayed by the “writeln” statements:

1. In main 2. In thread2 3. In thread1. A thread is only created when it is called, and the caller keeps running right away, so the parent prints its line before its newly created child gets the CPU.

LO2 and LO3: Compile and run the code; process list and process tree

ThreadTest1 was compiled and loaded in memory, and the console window was opened with Stay on top checked. One process of THREADTEST1 was created in the OS simulator, with Round Robin scheduling and the fastest speed, and START was clicked.

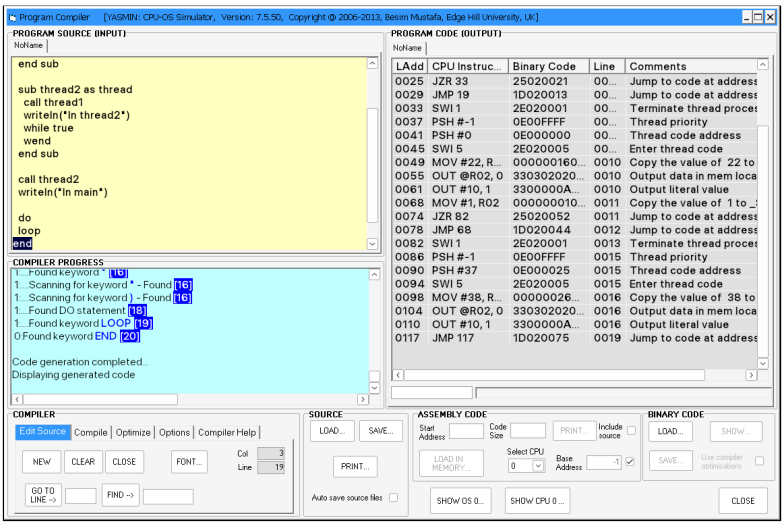


Figure 1. ThreadTest1 compiled in the Program Compiler.

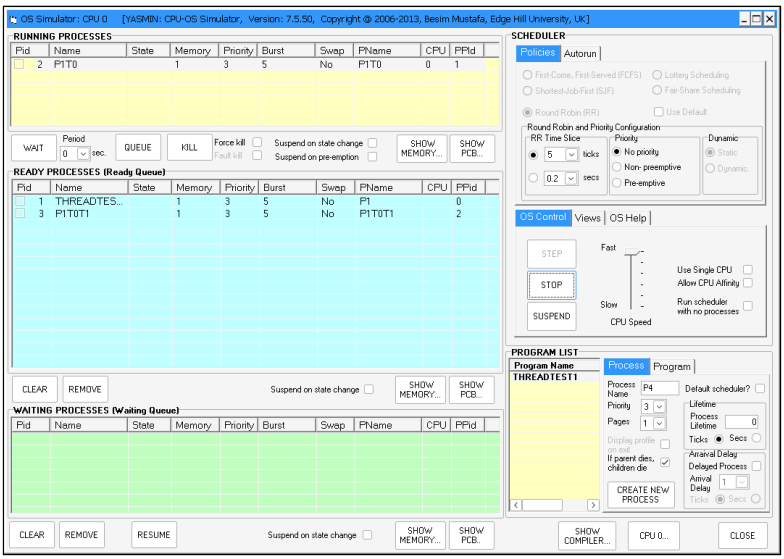


Figure 2. OS simulator after START: THREADTEST1, P1T0 and P1T0T1 sharing the CPU.

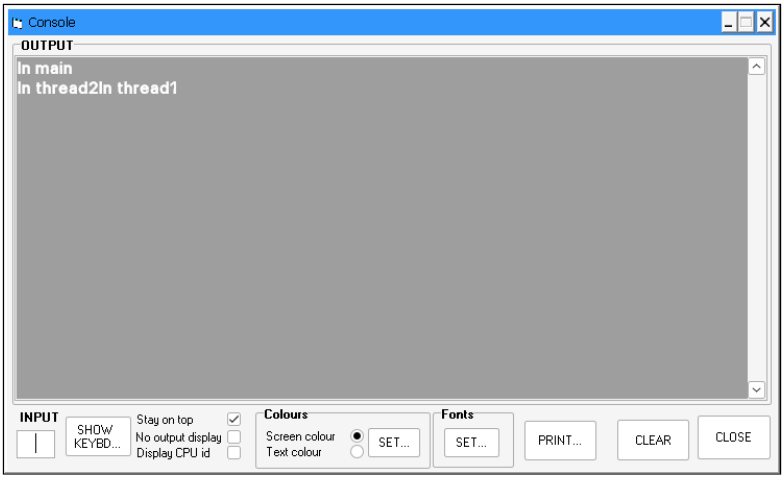


Figure 3. Console output of ThreadTest1.

Briefly explain your observations:

As soon as START was clicked the console showed “In main”, then “In thread2”, then “In thread1”, which is the order I expected. The last two ended up on one line (“In thread2In thread1”) because of Round Robin: thread2 lost the CPU after writing its text but before writing the new line, and thread1 printed in between. After that nothing else was printed, because all three keep running their endless loops and take turns on the CPU.

How many processes are created?

Three: one process and two threads.

Identify, by name, which is a process and which is a thread:

Name	PID	Type
THREADTEST1	1	Process (the main program)
P1T0	2	Thread (thread2, created by process 1)
P1T0T1	3	Thread (thread1, created by thread P1T0)

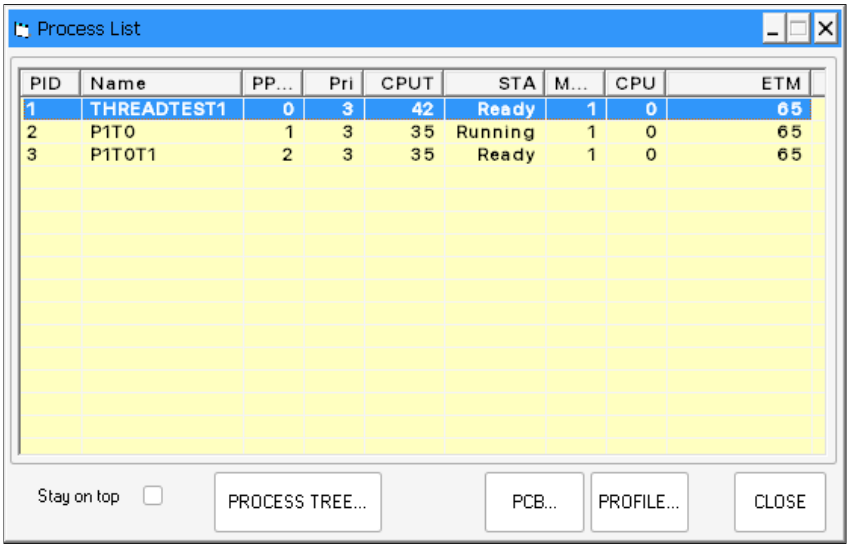


Figure 4. Process List window.

Briefly explain your observations (Process List):

The list shows all three entries: PID 1 THREADTEST1 (PPID 0), PID 2 P1T0 (PPID 1) and PID 3 P1T0T1 (PPID 2), with their priority (3), CPU time, state (Ready or Running), memory and CPU number. The CPU time of each keeps going up as they share CPU 0.

What do you think the PPID field signifies?

PPID is the Parent Process ID: the PID of the process that created it. P1T0 has PPID 1 because THREADTEST1 created it, and P1T0T1 has PPID 2 because P1T0 created it. THREADTEST1 has PPID 0 because it was created by the OS (the root).

Is there equivalent information in MS Windows?

Yes. Task Manager’s Details tab shows each process’s PID, and the number of threads can be added as a column. The parent process ID can be seen with tools like Process Explorer, or with the command

wmic process get Name,ProcessId,ParentProcessId (or Get-CimInstance Win32_Process in PowerShell).

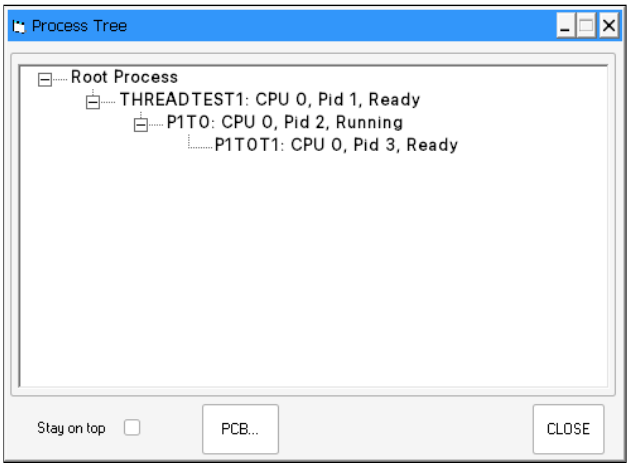


Figure 5. Process Tree window.

Briefly explain your observations (Process Tree):

The tree starts at “Root Process”. Under it is THREADTEST1 (Pid 1), under that is P1T0 (Pid 2), and under that is P1T0T1 (Pid 3). Each line also shows the CPU and the current state (Ready or Running).

How are the parent/child process relationships represented?

By indentation and connecting lines, like a folder tree: a child is drawn one level further right, directly under its parent, and a parent node can be expanded or collapsed with the +/– box.

Identify the parent and the children processes:

THREADTEST1 (Pid 1) is the parent of P1T0 (Pid 2). P1T0 is the parent of P1T0T1 (Pid 3). THREADTEST1 is therefore the grandparent of P1T0T1. The Root Process (the OS) is the parent of THREADTEST1.

Is there equivalent information in MS Windows?

Yes. Process Explorer (Sysinternals) shows a process tree with indentation. Task Manager also groups child processes under their app on the Processes tab, and Resource Monitor and the PowerShell Win32_Process data give the parent IDs.

The processes were then stopped with KILL.

LO4: Demonstrate that threads share their parent's data areas

```
program ThreadTest2
  var s1 string(6)
  var s2 string(6)

  sub thread1 as thread
    s1 = "hello1"
    writeln("In thread1")
    while true
    wend
  end sub

  sub thread2 as thread
    call thread1
    s2 = "hello2"
    writeln("In thread2")
    while true
    wend
  end sub

  call thread2
  writeln("In main")
  wait

  writeln(s1)
  writeln(s2)
end
```

Briefly explain what effect the modifications will have:

Two global string variables, s1 and s2, are declared in the main program. Thread1 stores "hello1" in s1 and thread2 stores "hello2" in s2. The main program now waits for its children to finish (wait) and then prints s1 and s2. If the threads share the parent's data, the parent will see "hello1" and "hello2" in its own variables. Because the threads loop forever, the wait never ends, so writeln(s1) and writeln(s2) are never reached.

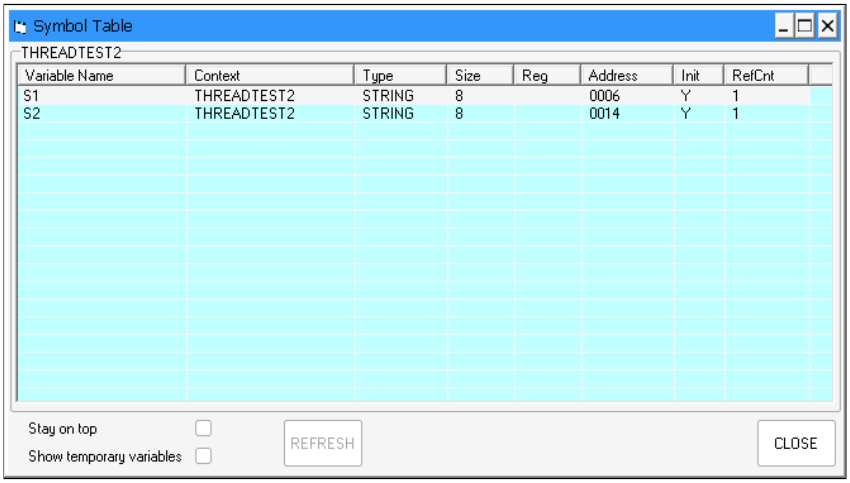


Figure 6. Symbol table of ThreadTest2.

Make a note below of the data memory addresses for variables s1 and s2:

s1: address 0006 s2: address 0014 (both STRING, size 8 bytes, context THREADTEST2)

The process was created and run as before (Round Robin, fastest). After all the displays, SUSPEND was clicked. The grandparent THREADTEST2 (Pid 1) was selected in the waiting queue and its memory was shown with SHOW MEMORY…; then the same was done for the two children.

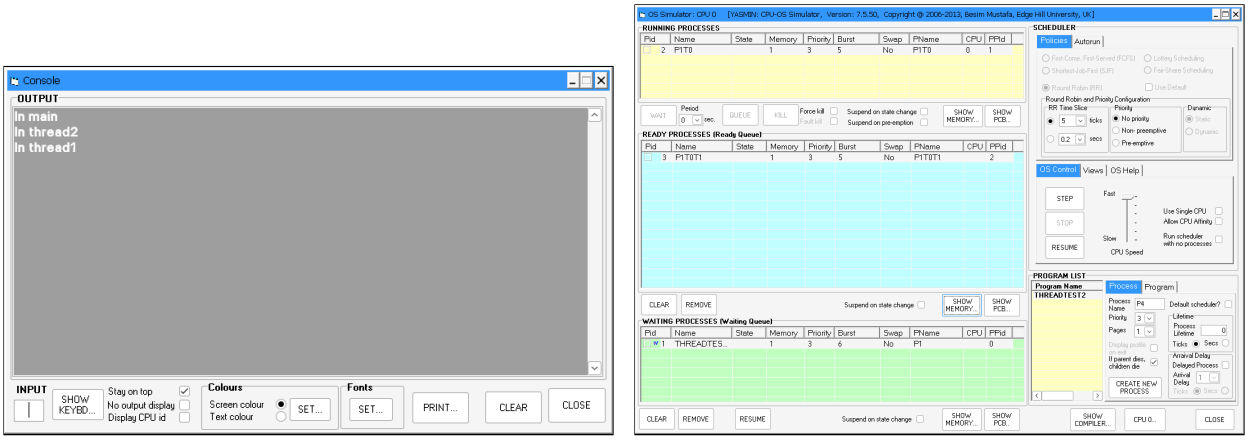


Figure 7. Console output (left) and the suspended OS simulator: THREADTEST2 waiting, P1T0 and P1T0T1 running and ready (right).

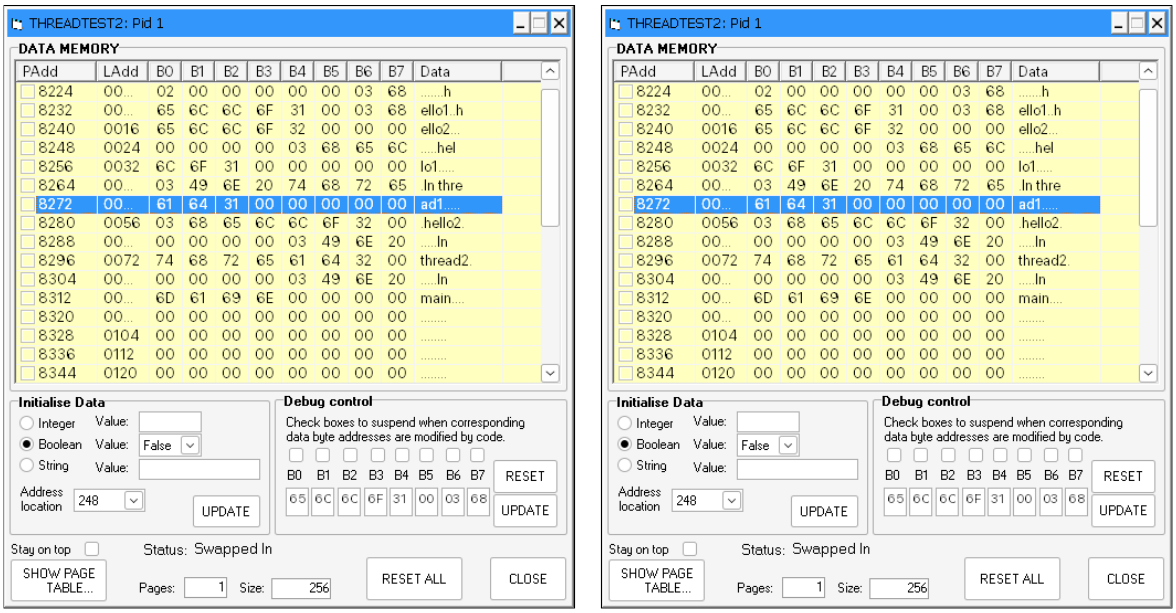


Figure 8. Data memory of the grandparent THREADTEST2 (left) and the window opened for child thread P1T0T1 (right).

Process selected	Window opened	Address 0006 (s1)	Address 0014 (s2)
THREADTEST2 (Pid 1, grandparent)	THREADTEST2: Pid 1	03 “hello1”	03 “hello2”
P1T0 (Pid 2, child)	THREADTEST2: Pid 1	03 “hello1”	03 “hello2”
P1T0T1 (Pid 3, grandchild)	THREADTEST2: Pid 1	03 “hello1”	03 “hello2”

Memory bytes read from the window: address 0006 = 03 68 65 6C 6C 6F 31 (03 then “hello1”), address 0014 = 03 68 65 6C 6C 6F 32 (03 then “hello2”).

Briefly explain what your general observations are and suggest what the significance of your observations is:

The grandparent THREADTEST2 never assigns s1 or s2 itself, yet its data memory holds “hello1” at s1 (0006) and “hello2” at s2 (0014). These values were written by thread1 and thread2. When the children were selected, SHOW MEMORY did not open a separate data area; it opened the same window, “THREADTEST2: Pid 1”, with the same bytes. So the threads have no data area of their own: they read and write the global variables of their parent (and grandparent) directly. This is why threads are called light-weight processes: creating one does not need a new copy of the data, and threads can share information through global variables without any special communication. The downside is that threads changing the same data at the same time can interfere with each other, so shared data must be protected (for example with a critical region) when needed.

RESUME was clicked and the processes were stopped with KILL.

LO5: A version not using threads

ThreadTest1 was copied as ThreadTest3 and the two “as thread” were removed:

```
program ThreadTest3
  sub thread1
    writeln("In thread1")
    while true
    wend
  end sub

  sub thread2
    call thread1
    writeln("In thread2")
    while true
    wend
  end sub

  call thread2
  writeln("In main")

  do
  loop
end
```

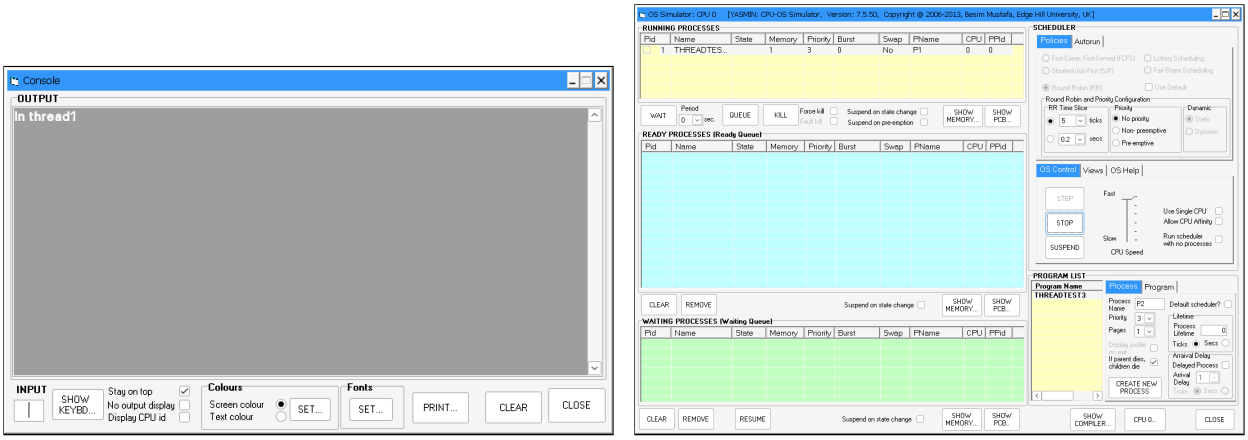


Figure 9. ThreadTest3: console output (left) and the OS simulator with a single process (right).

Briefly describe what you observe and explain how and why this differs from the results of the previous processes:

Only “In thread1” is displayed, and only one process (THREADTEST3, Pid 1) exists; no P1T0 or P1T0T1 is created. Without “as thread” the subroutines are ordinary subroutine calls that run inside the same process, one after the other. Main calls thread2, thread2 calls thread1 before printing anything, and thread1 prints “In thread1” and then enters its endless loop. Control never returns to thread2 or main, so “In thread2” and “In main” are never printed. With threads (ThreadTest1) each call created a separate thread that ran concurrently, so the caller could continue and all three messages appeared, in the order main, thread2, thread1, which is the reverse of the call nesting.

The process was stopped with KILL.

LO6: Simulate a multi-threaded server program

```
program ServerTest
  var p integer

  sub ServiceThread as thread
    writeln("Started service")
    for i = 1 to 30
    next
    writeln("Finished service")
  end sub

  sub ServerThread as thread
    while true
      read(nowait, p)
      select p
        case '1'
          call ServiceThread
        case 'q'
          break
        case else
      end select
    wend
  end sub

  call ServerThread
  wait
end
```

b. What you observe on the OS simulator window:

Two entries appear. SERVERTEST (Pid 1, the main program) goes straight into the WAITING queue, because after starting ServerThread it executes wait and waits for its child. P1T0 (Pid 2, the ServerThread) is in the RUNNING state. It runs its loop all the time, checking the keyboard with read(nowait, p) without blocking.

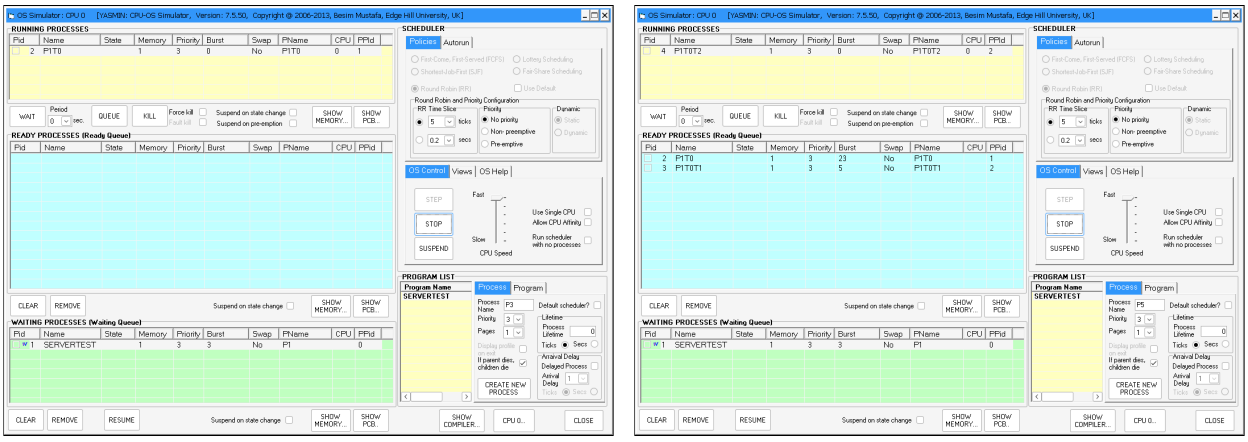


Figure 10. ServerTest after START (left) and after typing 1 (right): new service threads P1T0T1 and P1T0T2 appear next to the server thread P1T0.

c–d. Displays on the console after typing 1 four times and when all the threads finish:

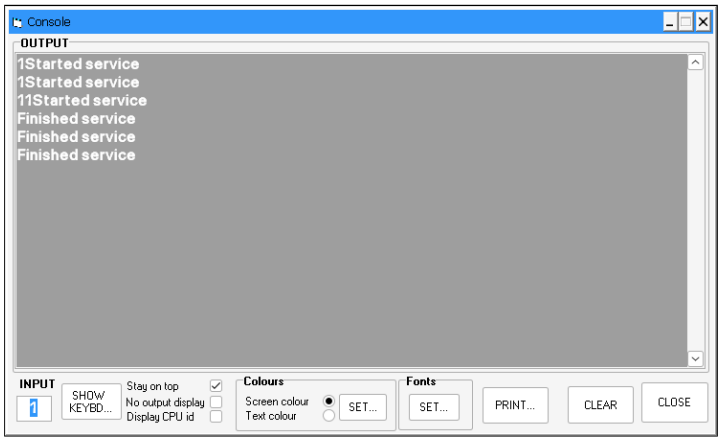


Figure 11. ServerTest console (threaded ServiceThread).

```
1Started service
1Started service
11Started service
Finished service
Finished service
Finished service
```

Each “1” is the key echoed by the console. Every key press makes the server create a new ServiceThread (P1T0T1, P1T0T2, …), and the server goes back to reading keys at once. All the “Started service” lines come first and the “Finished service” lines come after, because the service threads run at the same time. Only three services were started for four presses: two presses (“11”) arrived before the server thread got the CPU again, and since read(nowait, p) keeps only the last key, one press was lost. The same result came out in every repeat of the run. After “q” was typed the server left its loop, the main program finished its wait, and all processes ended.

ServiceThread was then changed to a normal subroutine (“as thread” removed), saved as ServerTest2 and run the same way.

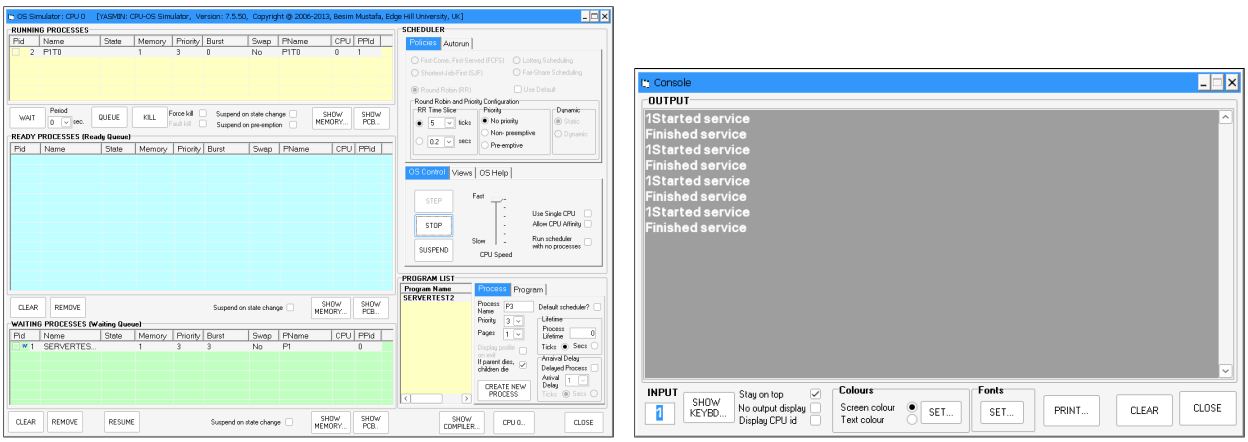


Figure 12. ServerTest2 (no service threads): only SERVERTEST2 and P1T0 exist (left); console (right).

h–i. Displays on the console after typing 1 four times and when all finish:

```
1Started service
Finished service
1Started service
Finished service
1Started service
Finished service
1Started service
Finished service
```

k. Do the displays differ? Explain why:

Yes, they differ. With ServiceThread as a thread, the server hands each request to a new thread and goes straight back to listening, so several services run at the same time and their output overlaps: all the "Started service" lines come before the "Finished service" lines. Without threads, the service runs inside the server thread itself, so the server must finish one request before it can read the next key. The output is strictly one "Started service" then its "Finished service", one request at a time, and no extra threads appear in the OS simulator. This is why real servers use threads: the server stays responsive and many clients are served together, instead of each client having to wait until the previous one is done.